

Name - Saikat Sheet
Batch - 25SUB5936_Linux System Programming
ID - 68500

Task 10:

Process Creation and Management in C

- Write a C program that demonstrates process creation and management:
- Use `fork()` to create a child process.
- In the child process, execute an external program (e.g., `ls` command) using `execvp()`.
- In the parent process, wait for the child process to complete and print a message indicating the child's termination.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // For fork(), execvp(), and getpid()
#include <sys/wait.h> // For wait()
#include <sys/types.h> // For pid_t

int main() {
    pid_t pid;

    // Use fork() to create a child process
    // fork() returns the Child's PID to the parent and 0 to the child
    pid = fork();

    if (pid < 0) {
        // Forking failed
        perror("Fork failed");
    }
}
```

```

        exit(1);
    }
    else if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("Child Process (PID: %d) is starting...\n", getpid());

        // Execute an external program (ls command) using execvp()
        // "ls" is the command, "-l" is the argument (long format)
        char *args[] = {"ls", "-l", NULL};

        printf("Child executing 'ls -l':\n");
        if (execvp(args[0], args) == -1) {
            perror("Exec failed");
            exit(1);
        }

        // This line is NEVER reached if execvp is successful
    }
    else {
        // --- PARENT PROCESS ---
        int status;
        printf("Parent Process (PID: %d) waiting for Child (PID: %d)...\n", getpid(),
pid);

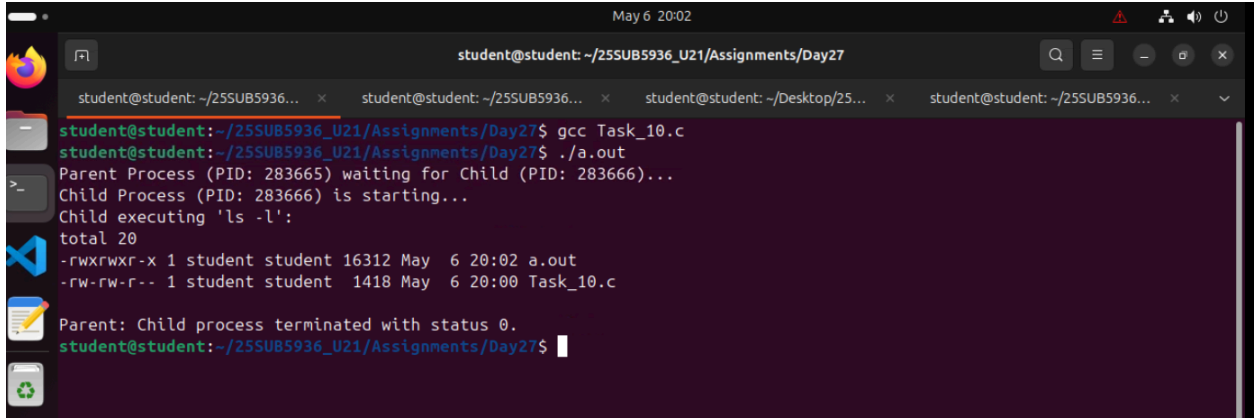
        // 3. Wait for the child process to complete
        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("\nParent: Child process terminated with status %d.\n",
WEXITSTATUS(status));
        }
    }

    return 0;
}

```

Output:



```
student@student: ~/25SUB5936_U21/Assignments/Day27
student@student: ~/25SUB5936_U21/Assignments/Day27$ gcc Task_10.c
student@student: ~/25SUB5936_U21/Assignments/Day27$ ./a.out
Parent Process (PID: 283665) waiting for Child (PID: 283666)...
Child Process (PID: 283666) is starting...
Child executing 'ls -l':
total 20
-rwxrwxr-x 1 student student 16312 May  6 20:02 a.out
-rw-rw-r-- 1 student student 1418 May  6 20:00 Task_10.c
Parent: Child process terminated with status 0.
student@student: ~/25SUB5936_U21/Assignments/Day27$
```